NDNSF-UAV-APP

Named Services for Secure Control, Adaptive Video, and Multi-Drone Operations

Tianxing Ma

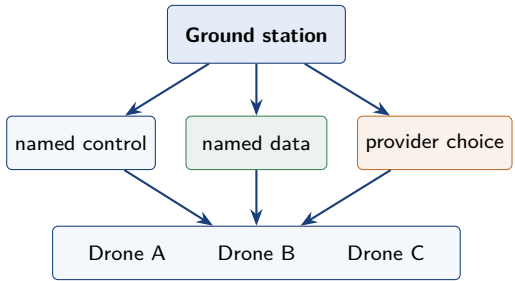
Department of Computer Science
University of Memphis

July 2026

Why an NDN-Native UAV Application?

One mission combines different communication needs

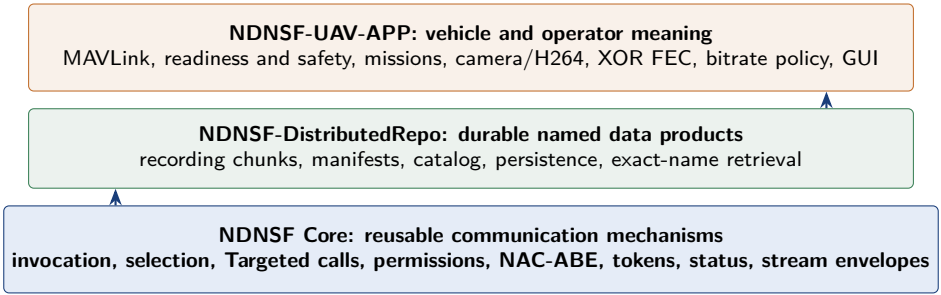
- Safety-sensitive commands need authorization and prompt failure.
- Telemetry and readiness must stay tied to the selected vehicle.
- Live video is continuous; recordings and logs are durable objects.
- Multiple mobile drones may be busy, disconnected, or replaceable.



The application binds UAV operations to **service and data names**, not to one fixed endpoint connection.

Evidence: NDNSF-UAV-APP/README.md: Why This Application Exists

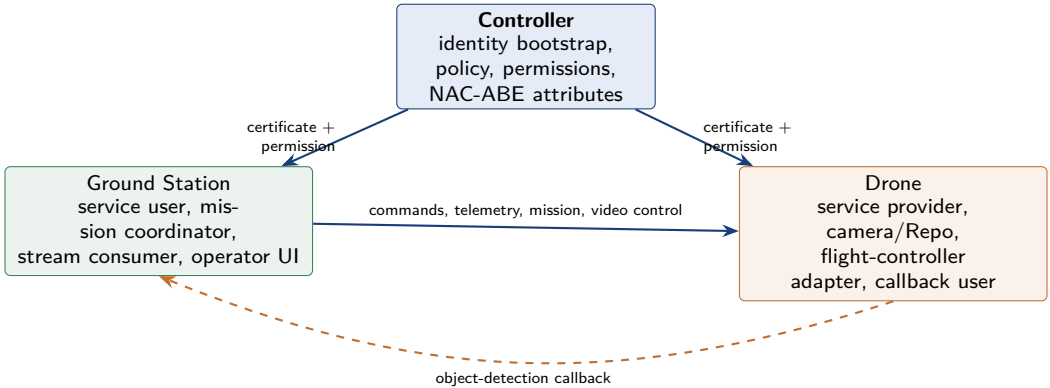
Responsibility Boundary



Core provides generic envelopes and security; the UAV application remains responsible for flight semantics and policy.

Evidence: docs/ndnsf-core-app-boundary.md; specs/070-uav-qgc-parity-boundary

Three Runtime Roles

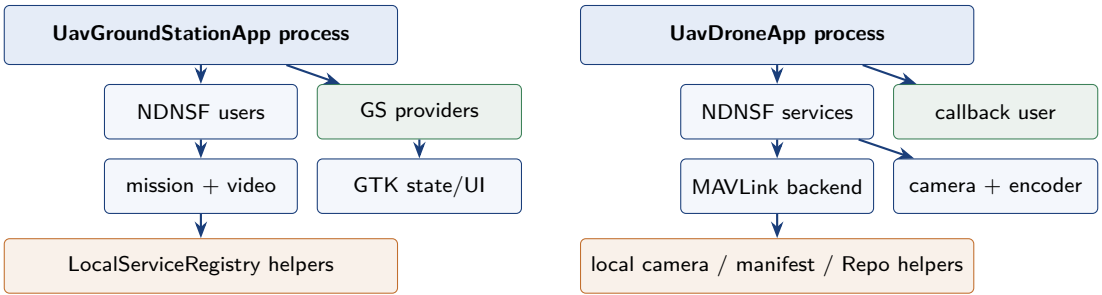


Typical PC: Controller + Ground Station + NFD

Each vehicle: DroneAPP + NFD + camera + MAVLink link

Evidence: UavDroneApp.cpp; UavGroundStationApp.cpp; README deployment section

Service Containers, Not One Monolithic RPC



One process shares identity, configuration, hardware, and lifecycle; every remote service remains separately named and permission-controlled.

Evidence: DroneServiceContainer.inc.hpp; GroundStationServiceContainer.inc.hpp

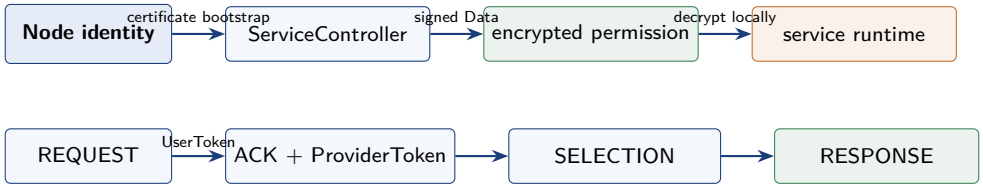
Named Services and Invocation Modes

Name	Mode	Purpose
/UAV/MAVLink/Execute	Targeted only	known-drone low-latency command path
/UAV/Telemetry/GetStatus	normal + Targeted	discovery or selected-drone state refresh
/UAV/Mission/Assign	normal	multi-provider mission-slot selection
/<drone>/UAV/Camera/Video	normal	provider-specific Start/Stop control
/<drone>/UAV/MAVLink/Parameters	normal	vehicle capability and parameter snapshot
/<drone>/UAV/Preflight/Checklist	normal	typed operator readiness details
/<drone>/UAV/Camera/Recording/Manifest	normal	discover durable recording chunks
/UAV/GS/ObjectDetection	normal	drone-to-ground-station compute callback

Shared service names allow provider selection; provider-specific names address one physical vehicle or its data product.

Evidence: shared/UavNames.hpp; DroneServiceContainer::installServiceInstances

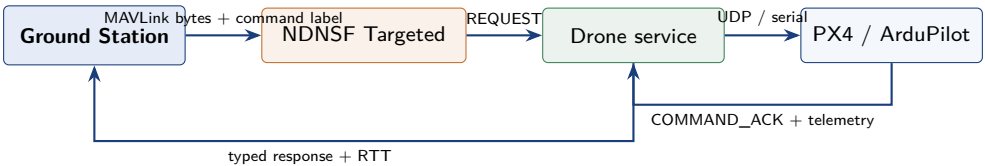
Secure Startup and Invocation



- REQUEST/SELECTION use service attributes; ACK/RESPONSE use permission attributes.
- One-time user/provider tokens bind the handshake and reject replay.
- Controller permission Data is signed and encrypted to the target identity certificate.

Evidence: ServiceController; ServiceUser/ServiceProvider security path; UAV deployment README

Targeted MAVLink Command Path



First use

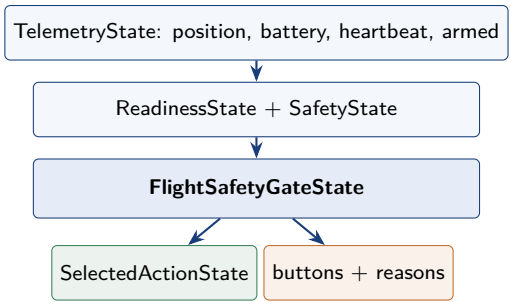
- authenticated ACK/selection bootstrap
- obtain a batch of one-time token pairs

Later commands

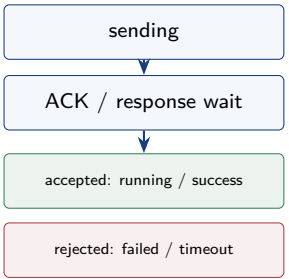
- request/response-only Targeted calls
- permissions, token checks, and replay protection remain

Evidence: sendMavlinkCommandToDrone; ServiceProvider TargetedOnly registration

Typed State Drives the Operator Workflow



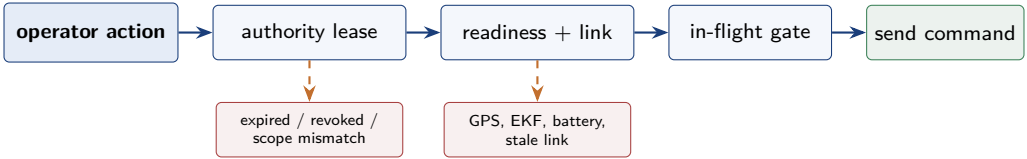
Command lifecycle



The GUI reads typed state. Stale or lost telemetry changes the same safety model used by the actual command-send path.

Evidence: shared/UavProtocol.hpp; GroundStationRuntimeState.hpp; commandLifecycle

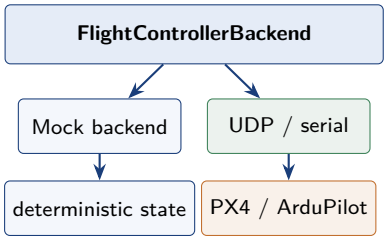
Safety Gates and Operator Authority



- A lease binds operator, drone set, monitor/control scope, and time window.
- Monitor authority permits observation but does not permit flight control.
- Authority never overrides readiness or safety; both gates must pass.
- Rejection reasons and authority alerts remain visible and auditable.

Evidence: OperatorAuthorityLease; validateOperatorLease; validateFlightSafetyGate

Flight-Controller Adapter Boundary



Backend contract

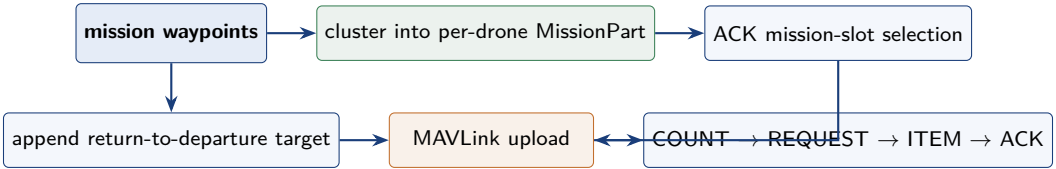
- forward MAVLink frames
- return command ACK fields
- expose latest telemetry
- read/edit parameters
- upload mission waypoints

Real-link helpers

- GCS heartbeat
- manual-control replay
- neutral command after timeout

Evidence: FlightControllerBackend; MockFlightControllerBackend; UdpFlightControllerBackend

Multi-Drone Mission Lifecycle

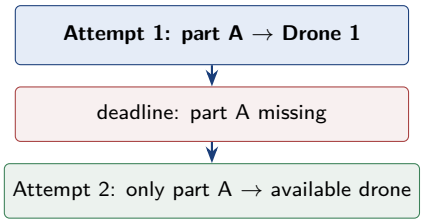


Mission start is phased: arm accepted drones, take off accepted drones, then start only the mission parts that were uploaded successfully.

Evidence: MissionPlan/MissionPart; uploadMissionPlan; executeMissionWaypoints

Mission Recovery and Persistence

Application-level compensation



- shared `patrol_task_id`; fresh NDNSF request each attempt
- no restart of already completed mission parts

Persistent mission document

- stable plan and operator identifiers
- per-drone parts and waypoint order
- geofence/rally metadata for future expansion
- save, load, preview, and upload through one typed model

Compensation repairs missing service results; it is not autonomous in-flight route optimization.

Evidence: MissionProgressState; MissionPlanDocument; runPatrolCompensationTask

Setup and Inspection Services

Parameters

- firmware, modes, completeness
- typed value and dry-run
- expected-value check
- read-back verification

Preflight

- named checklist items
- severity and blocking state
- operator-readable reason
- refresh before dangerous action

Analyze

- MAVLink message summaries
- active rates and last-seen state
- vehicle/mission snapshot
- inspector-oriented output

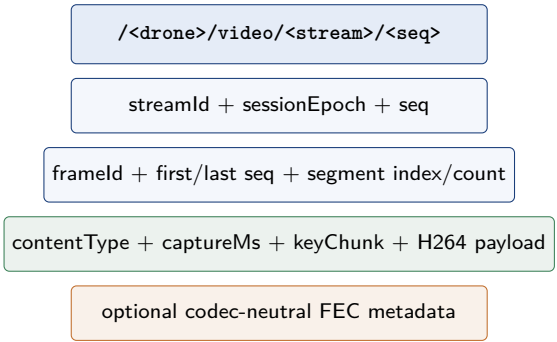
Evidence: VehicleParameterEditRequest/Result; PreflightCheckItem; UavAnalyzeSnapshot

Live Video: Control and Data Are Separate

- The service response starts or stops a session; it does not carry the video stream.
- Packet names are predictable and monotonically increasing within a stream.
- Stop is idempotent; late Interests and packets from the stopped session are ignored.

Evidence: VideoPublisher::start/stop; startVideoAttempt; README Video Streaming Design

Stream Packet and Session Model



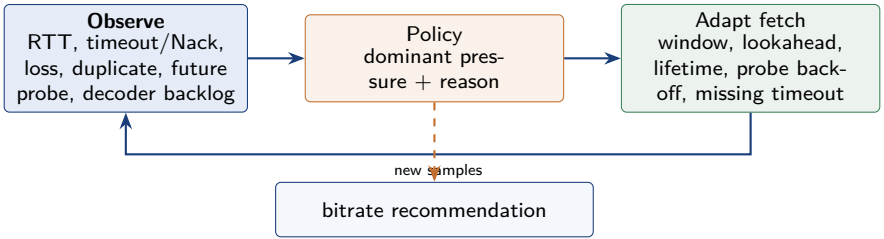
Consumer guards

- reject an old stream id or session epoch
- suppress duplicate packet sequence numbers
- reorder chunks before decoder input
- skip a late missing delta chunk to keep playback live

`VideoPacket` maps to core `StreamChunk` semantics without changing the existing UAV wire format.

Evidence: Stream.hpp; VideoPacket; videoPacketToStreamChunk; isCurrentVideoSessionPacket

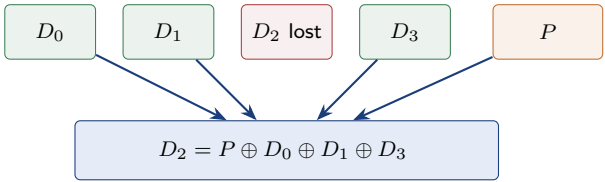
Adaptive Receiver Control Loop



Fetch adaptation is automatic. A producer bitrate change is explicit: Stop the old session, then Start a coherent new session with the requested bitrate.

Evidence: VideoAdaptivePolicyInput/Decision; requestVideoLane; restartVideoWithBitrateAfterStop

One-Loss XOR Forward Error Correction



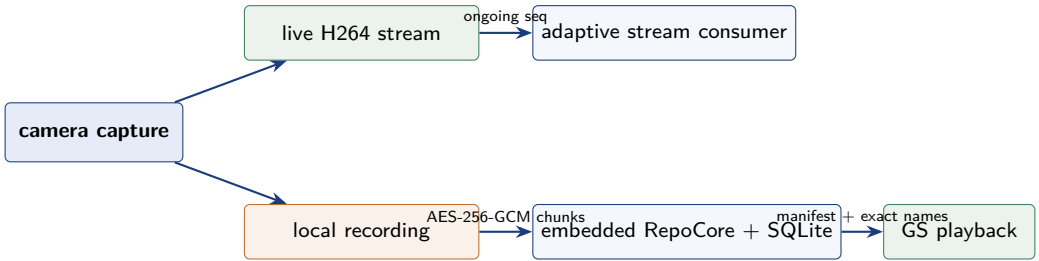
Current behavior

- one parity shard per frame by default
- original lengths carried in metadata
- recover before decoder insertion
- more than one missing data shard is not recoverable

FEC reduces retransmission delay for a single hole; reorder and stale-frame skipping preserve liveness when recovery is impossible.

Evidence: VideoPublisher::publishCurrentFrame; processFecChunk; attemptAndRecoverFrame

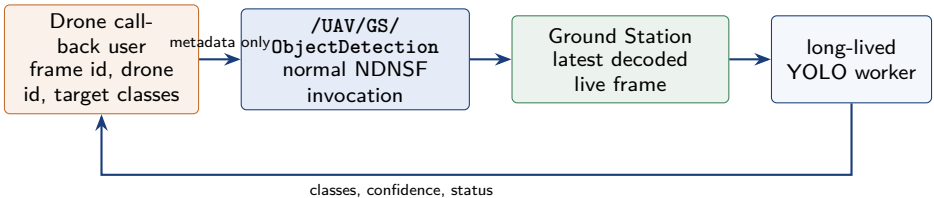
Recording Is a Named Object, Not a Live Stream



- Capture, recording, and viewing have independent lifecycles.
- The manifest and catalog discover durable chunks under publisher-owned names.
- Playback fetches and decrypts named chunks; it does not continue the live stream session.

Evidence: VideoPublisher recording path; RepoCore; Ground Station recording playback

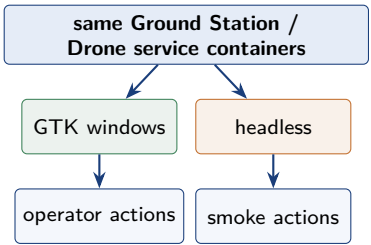
Ground-Station Object Detection Callback



Roles can reverse: a drone is normally a provider, but it can consume a heavier service from the ground station. Image bytes are reused locally and are not inserted into the service request.

Evidence: installObjectDetectionService; startObjectDetectionLoop; yolo_detect_worker.py

One Runtime, GUI or Headless



Validation layers

- C++ protocol/state unit tests
- deployment and config quick smoke
- MiniNDN multi-node service tests
- PX4 SITL + jMAVSim command/mission tests
- GUI smoke under Xvfb

Default MiniNDN placement

- Controller + GS: Memphis
- Drone A: UCLA
- Drone B: WUSTL

Evidence: MiniNDN UAV launcher; UAV protocol/state unit tests

Implemented Foundation and Current Boundary

Implemented foundation

- named, permission-controlled UAV services
- Targeted command path and typed control state
- operator authority and safety gates
- multi-drone mission assignment and compensation
- adaptive H264 delivery with session guards and XOR FEC
- encrypted recording discovery and playback

Not yet a production ground station

- complete calibration and firmware setup
- full survey/geofence/altitude-profile editing
- hardened lost-link and autopilot fail-safe policy
- long-duration outdoor and hardware-breadth evidence
- certified operator workflow and safety assurance

Next: use repeated real-device campaigns to harden safety and recovery, while moving only genuinely app-neutral mechanisms back into NDNSF Core.

Evidence: NDNSF-UAV-APP/README.md: Current Boundary and Comparison With QGroundControl